



BURSA TEKNİK ÜNİVERSİTESİ

**MÜHENDİSLİK VE DOĞA BİLİMLERİ FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ
NESNEYE YÖNELİK PROGRAMLAMA DERSİ
BANKA OTOMASYON SİSTEMİ PROJE RAPORU**

AD:ELİF NİSA

SOYAD:YÜKSEL

ÖĞRENCİ NUMARASI: 25360859092

İÇİNDEKİLER

1. Giriş

1.1 Projenin Amacı ve Kapsamı

1.2 GitHub Reposu ve Youtube Linki

2. Programın Teknik Detayları

2.1 Programın Akışı

2.2 Paket Yapısı ve Modülerlik

2.3 Kalıtım ve Sınıf Hiyerarşisi

2.4 Otomatik Veri Üretimi ve Tip Yönetimi

2.5 Dinamik Veri Yapıları

3. Programın Ekran Çıktıları

3.1. Başarılı İşlem Senaryosu

3.2. Başarısız İşlem Senaryoları ve Hata Yönetimi

4. Sonuç

5. Kaynakça

1. Giriş

1.1 Projenin Amacı ve Kapsamı

Java programlama dili kullanılarak gerçekleştirilen bu projede, bankacılık sektöründeki temel operasyonel süreçlerin nesneye yönelik programlama (NYP) prensipleri çerçevesinde yazılım ortamına taşınması amaçlanmaktadır. İnsan gücüyle takibi ve yönetimi karmaşık olan müşteri, hesap ve kredi kartı verilerinin bilgisayar sistemleri aracılığıyla güvenli, hızlı ve hatasız bir şekilde işlenmesi hedeflenmiştir. Proje kapsamında; kalıtım, kapsülleme ve polimorfizm gibi NYP kavramları kullanılarak modüler bir yapı oluşturulmuş; vadesiz ve yatırım hesapları üzerinden finansal işlemler ile kredi kartı limit/borç yönetimi simüle edilmiştir. Mevcut çalışma, bankacılık sisteminin temel fonksiyonlarıyla sınırlandırılmış olsa da esnek paket yapısı sayesinde ilerleyen süreçlerde yeni özellikler eklenerek genişletilmeye uygun bir mimariye sahiptir. Bu proje, Bursa Teknik Üniversitesi Bilgisayar Mühendisliği bölümü Nesneye Yönelik Programlama dersi için bireysel bir çalışma olarak geliştirilmiştir.

1.2 GitHub Reposu ve Youtube Linki

Projenin yüklendiği GitHub reposu:

https://github.com/elifnisayuksel7-a11y/NYP_proje/tree/main

Youtube videosu linki:

<https://www.youtube.com/watch?v=M99ERWk7OY0>

2. Programın Teknik Detayları

2.1 Programın Akışı

Geliştirilen bankacılık simülasyonu, nesne yönelimli programlama prensiplerine uygun olarak modüler bir hiyerarşi içinde çalışmaktadır. Programın çalışma akışı ve metotların tetiklenme sırası şu şekildedir:

- **Sistem Yetkilisinin Tanımlanması:** Program başlatıldığında ilk olarak `com.bank.app.people` paketi altındaki `BankaPersoneli` sınıfından bir nesne üretilerek sisteme giriş yapan personel tanımlanır. Bu aşamada personelin kişisel bilgileri ve sistem tarafından otomatik atanan benzersiz `personeIID` bilgisi konsola yazdırılır.

- **Müşteri Kaydı ve İlişkilendirme:** Yeni bir *Musteri* nesnesi oluşturularak sistemdeki personelin *musteriler* isimli *ArrayList* yapısına eklenir. Böylelikle personel ve sorumlu olduğu müşteri arasında nesne düzeyinde bir bağlantı kurulur.

- **Dinamik Hesap ve Kart Tanımlama:** Kaydı yapılan müşteri için *hesapEkle* metodu kullanılarak çalışma zamanında (runtime) "Vadesiz" ve "Yatırım" türünde iki farklı hesap açılır. Ardından *krediKartiEkle* metodu ile müşteriye özel bir limit tanımlanarak *KrediKarti* nesnesi oluşturulur ve ilgili listelere (*ArrayList*) dahil edilir.

- **Finansal Simülasyon ve İşlem Geçmişi:**

- ***Yatırım İşlemleri:** *YatirimHesabi* sınıfındaki *paraEkle* ve *paraCek* metotları test edilerek bakiyenin güncelliği ve işlem doğruluğu kontrol edilir.

- ***Harcama Yönetimi:** Kredi kartı nesnesi üzerinden *alisverisYap* metodu çağrılarak güncel borç ve kullanılabilir limit değişimleri gözlemlenir.

- ***Borç Ödeme Dinamiği:** *VadesizHesap* sınıfındaki *krediKartiBorcOde* metodu aracılığıyla, hesaptaki bakiye kullanılarak kredi kartı borcu kapatılır. Bu işlem sırasında hem hesap bakiyesinin azalması hem de kart borcunun düşmesi sağlanarak sınıflar arası veri tutarlılığı korunur.

- **Güvenlik ve Kontrol Mekanizmaları:** Proje kapsamında geliştirilen *hesapSil* ve *krediKartiSil* metotları ile sistemin güvenliği test edilir. Bakiyesi 0'dan büyük olan hesapların veya borcu bulunan kartların silinmesi engellenerek kullanıcıya gerekli uyarı mesajlarının gösterilmesi sağlanır.

- **Durum Raporlama:** Programın sonunda, yapılan tüm işlemlerin neticesini ve son bakiyeleri göstermek amacıyla tüm nesnelerin *toString()* metodları çağrılarak güncel durum özeti kullanıcıya sunulur.

2.2 Paket Yapısı ve Modülerlik

Proje, Nesne Yönelik Programlama prensiplerine uygun olarak beş ana paket altında organize edilmiştir. *com.bank.app.people* paketi kullanıcı sınıflarını, *com.bank.app.accounts* ve *com.bank.app.cards* paketleri finansal araçları, *com.bank.app.service* paketi iş mantığını, *com.bank.app.main* ise uygulama akışını içermektedir. Bu yapı, kodun okunabilirliğini ve sürdürülebilirliğini artırmaktadır.

2.3 Kalıtım ve Sınıf Hiyerarşisi

Projenin temelini oluşturan *Kisi* sınıfı; *ad*, *soyad*, *email* ve *telefonNumarasi* gibi ortak özellikleri barındırmaktadır. *Musteri* ve *BankaPersoneli* sınıfları bu sınıftan miras alarak kod tekrarını önlemiştir. Benzer şekilde *BankaHesabi* sınıfından türetilen *VadesizHesap* ve *YatirimHesabi* sınıfları, bankacılık işlemlerinde polimorfik bir yapı sunmaktadır.

2.4 Otomatik Veri Üretimi ve Tip Yönetimi

personelID, *musteriNumarasi*, *kartNumarasi* ve *iban* gibi benzersiz değerler, *Random* kütüphanesi kullanılarak çalışma zamanında otomatik olarak üretilmektedir. Ayrıca, Java'nın *int* veri tipinin kapasitesini aşan telefon numarası gibi veriler, veri kaybını önlemek amacıyla *long* tipiyle yönetilmiş ve tip güvenliği sağlanmıştır.

2.5 Dinamik Veri Yapıları

Müşterilerin sahip olduğu birden fazla hesap ve kredi kartını, ayrıca banka personelinin sorumlu olduğu müşteri listelerini tutmak amacıyla Java'nın dinamik *ArrayList* yapısı tercih edilmiştir. Bu sayede sistem, esnek bir veri depolama imkanına sahip olmuştur.

3. Programın Ekran Çıktıları

3.1. Başarılı İşlem Senaryosu

Proje Yapısı ve Nesne Oluşturma

Sistemin ilk aşamasında, banka çalışanlarının sisteme dahil olması için interaktif bir giriş paneli tasarlanmıştır. Bu panel üzerinden alınan veriler, nesneye dayalı programlama prensipleri doğrultusunda *BankaPersoneli* sınıfından bir nesne üretilmesini sağlamaktadır.

```
***** BANKA YÖNETİM SİSTEMİ *****
--- Personel Giriş Paneli ---
Personel Adı: Büşra
Personel Soyadı: Yüksel
Email: busra@gmail.com
Telefon: 5434497279

Personel Girişi Başarılı.
Ad: Büşra, Soyad: Yüksel, Email: busra@gmail.com, Telefon: 5434497279, Personel ID: 527170
```

Personel girişinin ardından, bankacılık hizmetlerinden yararlanacak olan müşteri nesnesinin oluşturulma süreci test edilmiştir. Sistem, her yeni müşteri için benzersiz bir kimlik numarası üreterek veri tabanı tutarlılığını simüle etmektedir.

```
--- Yeni Müşteri Kayıt Paneli ---
Müşteri Adı: Elif
Müşteri Soyadı: Yüksel
Email: elif@gmail.com
Telefon: 5356482589

Müşteri Başarıyla Kaydedildi.
Ad: Elif, Soyad: Yüksel, Email: elif@gmail.com, Telefon: 5356482589, Müşteri No: 919750
```

Hesap İşlemleri ve Bakiye Yönetimi

Bu bölümde, oluşturulan hesaplara dinamik olarak bakiye tanımlanması ve hesapların türlerine göre verdikleri çıktı yanıtları test edilmiştir.

Sistem, kullanıcıdan alınan girdi miktarlarını ilgili hesap nesnelerine aktararak bakiye güncellemelerini gerçekleştirmektedir.

```
--- Bakiye İşlemleri ---
Yatırım hesabına yatırılacak tutar: 1000
[Yatırım Hesabı] 1000.0 TL para yatırma işlemi başarılı. Güncel bakiye: 1000.0
Vadesiz hesaba yatırılacak tutar: 5000
```

Hesaplar Arası Para Transferi

Sistemin finansal tutarlılığını test etmek amacıyla, vadesiz hesap ile yatırım hesabı arasında interaktif bir para aktarımı gerçekleştirilmiştir. Bu işlem, nesneye dayalı programlamada nesnelerin birbiriyle etkileşim kurma ve veri paylaşma yeteneğini temsil etmektedir.

```
--- Transfer İşlemi ---
Vadesizden Yatırıma aktarılacak miktar: 2000
[Vadesiz Hesap] Transfer başarılı. Alıcı IBAN: TR46207996
```

Sistem, kaynak hesabın bakiyesini kontrol etmiş ve yeterli bakiye olması durumunda tutarı bu hesaptan düşmüştür. Ardından, alıcı hesabın (Yatırım Hesabı) referansını kullanarak ilgili tutarı alıcı bakiyesine eklemiştir.

Kredi Kartı Harcama ve Limit Yönetimi

Sistemin kredi kartı modülü kapsamında, kullanıcı tarafından belirlenen limitler dahilinde harcama yapabilme yeteneği test edilmiştir. Bu bölüm, kart limitinin harcama sonrası otomatik olarak güncellenmesini ve borç miktarının takibini simüle etmektedir.

```
--- Kredi Kartı İşlemleri ---
Kredi kartı limiti ne kadar olsun?: 5000
Yapılacak alışveriş tutarı: 2000
2000.0 TL tutarında alışveriş yapıldı
```

Kredi Kartı Borç Ödeme ve Bakiyesi Kontrolü

Kredi kartı modülünün ikinci aşamasında, vadesiz hesaptaki bakiye ile kart borcunun kapatılması süreci test edilmiştir. Bu bölüm, iki farklı nesne (*VadesizHesap* ve *KrediKarti*) arasındaki etkileşimin finansal kurallar çerçevesinde nasıl yönetildiğini göstermektedir.

```
Borç Ödeme Deneniyor...
2000.0 TL borç ödemesi yapıldı
[Vadesiz Hesap] Kart borcunuz başarıyla ödendi.
```

Kritik İşlem Kısıtlamaları ve Hesap Silme Kontrolü

Projenin güvenlik protokolleri gereği, banka hesaplarının silinmesi süreci belirli mantıksal kurallara bağlanmıştır. Bu bölümde, sistemin "bakiyesi olan bir hesabın silinmesini engelleme" yeteneği ve bu kuralın veri tutarlılığı üzerindeki etkisi test edilmiştir.

```
--- Hesap Silme İşlemi ---
Bakiyesi olan TR46207996 siliniyor...
Lütfen öncelikle bakiyenizi başka bir hesaba aktarınız.
Bakiyesi 0 olan yeni bir hesap açılıp siliniyor...
Hesap başarıyla silindi.
```

Güncel Durum Özeti ve Sistem Doğrulaması

Programın tüm iş akışı tamamlandıktan sonra, sistemdeki tüm varlıkların (hesaplar ve kartlar) son durumları toplu bir şekilde ekrana yansıtılmıştır.

```
--- Güncel Durum Özeti ---
IBAN: TR64432285 | Bakiye: 1000.0 TL
IBAN: TR46207996 | Bakiye: 3000.0 TL
Kart No: 5220142759310511 | Borç: 0.0 TL | Kullanılabilir Limit: 5000.0 TL

***** İŞLEMLER TAMAMLANDI *****
```


3.2.Başarısız İşlem Senaryoları ve Hata Yönetimi

Yetersiz Bakiye ile Para Transferi Denemesi

Sistemin finansal kararlılığını ve mantıksal sınırlarını test etmek amacıyla, hesapta bulunan mevcut bakiyenin üzerinde bir tutar ile para transferi gerçekleştirilmeye çalışılmıştır. Bu senaryo, sistemin kullanıcı hatalarına karşı gösterdiği direnci ve veri güvenliği protokollerini doğrulamaktadır.

```
--- Bakiye İşlemleri ---
Yatırım hesabına yatırılacak tutar: 500
[Yatırım Hesabı] 500.0 TL para yatırma işlemi başarılı. Güncel bakiye: 500.0
Vadesiz hesaba yatırılacak tutar: 500

--- Transfer İşlemi ---
Vadesizden Yatırıma aktarılacak miktar: 2000
[Vadesiz Hesap] Hata: Gönderen hesabın bakiyesi yetersiz!
```

Borç Ödemesi İçin Yetersiz Bakiye Kontrolü

Sistemin farklı modülleri arasındaki entegrasyonun güvenilirliğini ölçmek amacıyla, vadesiz hesaptaki yetersiz bakiye ile kredi kartı borcu ödeme işlemi simüle edilmiştir. Bu test, *KrediKarti* ve *VadesizHesap* sınıfları arasındaki bağımlı işlem mantığını ve nesne niteliklerinin korunmasını doğrulamaktadır.

```
--- Bakiye İşlemleri ---
Yatırım hesabına yatırılacak tutar: 500
[Yatırım Hesabı] 500.0 TL para yatırma işlemi başarılı. Güncel bakiye: 500.0
Vadesiz hesaba yatırılacak tutar: 500

--- Transfer İşlemi ---
Vadesizden Yatırıma aktarılacak miktar: 2000
[Vadesiz Hesap] Hata: Gönderen hesabın bakiyesi yetersiz!

--- Kredi Kartı İşlemleri ---
Kredi kartı limiti ne kadar olsun?: 5000
Yapılacak alışveriş tutarı: 3000
3000.0 TL tutarında alışveriş yapıldı

Borç Ödeme Deneniyor...
[Vadesiz Hesap] Hata: Hesabınızda borç ödemesi için yeterli bakiye bulunmamaktadır.
```

Geçersiz (Negatif) Tutar Girişi Testi

Sistemin finansal tutarlılığını korumak ve mantıksal olarak imkansız olan veri girişlerini engellemek amacıyla "negatif tutar" senaryosu test edilmiştir.

```
--- Bakiye İşlemleri ---  
Yatırım hesabına yatırılacak tutar: -500  
Hata: Yatırılacak tutar negatif veya sıfır olamaz!
```

SONUÇ

Bu çalışma kapsamında geliştirilen banka otomasyon sistemi, Java programlama dilinin nesneye yönelik (OOP) imkanları üzerine inşa edilmiştir. Projenin temelini oluşturan kalıtım, kapsülleme ve çok biçimlilik prensipleri; kodun modüler ve yönetilebilir olmasını sağlamıştır.

Uygulama aşamasında, vadesiz ve yatırım hesapları arasındaki transfer süreçleri ile otomatik IBAN ve kart numarası üretimi gibi operasyonel işlemler önceliklendirilmiştir. Özellikle sistem güvenliğini artırmak adına, içinde bakiye bulunan hesapların silinmesi veya borçlu kartların iptal edilmesi gibi kritik işlemler kısıtlanarak veri tutarlılığı güvence altına alınmıştır.

Teknik altyapıda `ArrayList` gibi dinamik yapılar tercih edilerek veri yönetimi esnekleştirilmiş, *Random* sınıfı ile benzersiz kimlik numaraları atanmıştır. Paket yapısının düzenli kurgulanması, projenin okunabilirliğini ve geliştirilebilirliğini artırmıştır. Sonuç olarak sistem, hedeflenen tüm bankacılık senaryolarını başarıyla karşılamaktadır.

KAYNAKÇA

- Yapay Zeka Araçları (ChatGPT, Gemini)
- Liang, Y. D. (2020). *Introduction to Java Programming and Data Structures* (12. bs.). Pearson
- Oracle Java Documentation (docs.oracle.com)